My Project

Generated by Doxygen 1.8.17

1 Class Index 1

1.1 Class List . . . 1

2 File Index 3

2.1 File List . . . 3

3 Class Documentation 5

3.1 sprite_s Struct Reference . . . 5

3.1.1 Detailed Description . . . 5

3.1.2 Member Data Documentation . . . 5

3.1.2.1 h . . . 5

3.1.2.2 w . . . 5

3.1.2.3 x . . . 6

3.1.2.4 y . . . 6

3.2 textures_s Struct Reference . . . 6

3.2.1 Detailed Description . . . 6

3.2.2 Member Data Documentation . . . 6

3.2.2.1 background . . . 6

3.2.2.2 ligne_arrivee . . . 7

3.2.2.3 meteorite . . . 7

3.2.2.4 vaisseau . . . 7

3.3 world_s Struct Reference . . . 7

3.3.1 Detailed Description . . . 8

3.3.2 Member Data Documentation . . . 8

3.3.2.1 gameover . . . 8

3.3.2.2 ligne_arrivee . . . 8

3.3.2.3 time . . . 8

3.3.2.4 vaisseau . . . 8

3.3.2.5 vy . . . 8

3.3.2.6 wall . . . 9

3.3.2.7 win . . . 9

4 File Documentation 11

4.1 constantes.h File Reference . . . 11

4.1.1 Detailed Description . . . 11

4.2 main.c File Reference . . . 12

4.2.1 Detailed Description . . . 12

4.2.2 Function Documentation . . . 13

4.2.2.1 clean() . . . 13

4.3 sdl2-light.c File Reference . . . 13

4.3.1 Detailed Description . . . 14

4.3.2 Function Documentation . . . 14

4.3.2.1 apply_texture() . . . 14

4.3.2.2 clean_sdl() . . . 15
4.3.2.3 clean_texture() . . . 15
4.3.2.4 clear_renderer() . . . 15
4.3.2.5 init_sdl() . . . 16
4.3.2.6 load_image() . . . 16
4.3.2.7 pause() . . . 16
4.3.2.8 update_screen() . . . 17
4.4 sdl2-light.h File Reference . . . 17
4.4.1 Detailed Description . . . 18
4.4.2 Function Documentation . . . 19
4.4.2.1 apply_texture() . . . 19
4.4.2.2 clean_sdl() . . . 19
4.4.2.3 clean_texture() . . . 20
4.4.2.4 clear_renderer() . . . 20
4.4.2.5 init_sdl() . . . 20
4.4.2.6 load_image() . . . 21
4.4.2.7 pause() . . . 21
4.4.2.8 update_screen() . . . 21

Index **23**

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

sprite_s
: Type qui correspond au sprite 5

textures_s
: Représentation pour stocker les textures nécessaires à l'affichage graphique 6

world_s
: Représentation du monde du jeu 7

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

affichage.h . . . . . **??**
constantes.h
Contient l'ensemble des constantes du jeu 11
logique.h . . . . . **??**
main.c
Programme principal du jeu 12
sdl2-light.c
Sur-couche de SDL2 pour simplifier son utilisation pour le projet 13
sdl2-light.h
En-tête du module correspondant à une sur-couche de SDL2 pour simplifier son utilisation pour le projet 17
sdl2-ttf-light.h . . . . . **??**

Chapter 3

Class Documentation

3.1 sprite_s Struct Reference

Type qui correspond au sprite.

`#include <affichage.h>`

Public Attributes

- float x
- float y
- float w
- float h

3.1.1 Detailed Description

Type qui correspond au sprite.

3.1.2 Member Data Documentation

3.1.2.1 h

`float sprite_s::h`

Champ indiquant la hauteur h du sprite

3.1.2.2 w

`float sprite_s::w`

Champ indiquant la largeur w du sprite

3.1.2.3 x

```
float sprite_s::x
```

Champ indiquant la position x du sprite

3.1.2.4 y

```
float sprite_s::y
```

Champ indiquant la position y du sprite

The documentation for this struct was generated from the following file:

- affichage.h

3.2 textures_s Struct Reference

Représentation pour stocker les textures nécessaires à l'affichage graphique.

```
#include <affichage.h>
```

Public Attributes

- SDL_Texture * background
- SDL_Texture * vaisseau
- SDL_Texture * ligne_arrivee
- SDL_Texture * meteorite

3.2.1 Detailed Description

Représentation pour stocker les textures nécessaires à l'affichage graphique.

3.2.2 Member Data Documentation

3.2.2.1 background

```
SDL_Texture* textures_s::background
```

Texture liée à l'image du fond de l'écran.

3.2.2.2 ligne_arrivee

```
SDL_Texture* textures_s::ligne_arrivee
```

Texture liée à l'image de la ligne d'arrivée

3.2.2.3 meteorite

```
SDL_Texture* textures_s::meteorite
```

Texture liée à l'image d'une météorite

3.2.2.4 vaisseau

```
SDL_Texture* textures_s::vaisseau
```

Texture liée à l'image du vaisseau.

The documentation for this struct was generated from the following file:

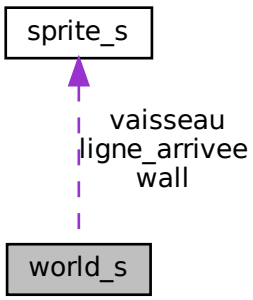
- affichage.h

3.3 world_s Struct Reference

Représentation du monde du jeu.

```
#include <affichage.h>
```

Collaboration diagram for world_s:



Public Attributes

- sprite_t vaisseau
- sprite_t ligne_arrivee
- sprite_t wall [6]
- float vy
- int gameover
- int win
- float time

3.3.1 Detailed Description

Représentation du monde du jeu.

3.3.2 Member Data Documentation

3.3.2.1 gameover

```
int world_s::gameover
```

Champ indiquant si on a perdu

3.3.2.2 ligne_arrivee

```
sprite_t world_s::ligne_arrivee
```

Champ représentant le sprite de la ligne d'arrivée

3.3.2.3 time

```
float world_s::time
```

Champ représentant le temps écoulé depuis le début du jeu

3.3.2.4 vaisseau

```
sprite_t world_s::vaisseau
```

Champ représentant le sprite du vaisseau

3.3.2.5 vy

```
float world_s::vy
```

Champ représentant la vitesse de déplacement de la ligne d'arrivée

3.3.2.6 wall

```
sprite_t world_s::wall[6]
```

Champ représentant le tableau du sprite des murs de météorites

3.3.2.7 win

```
int world_s::win
```

Champ indiquant si on a gagné

The documentation for this struct was generated from the following file:

- affichage.h

Chapter 4

File Documentation

4.1 constantes.h File Reference

Contient l'ensemble des constantes du jeu.

Macros

- #define SCREEN_WIDTH 300

  Largeur de l'écran de jeu.
- #define SCREEN_HEIGHT 480

  Hauteur de l'écran de jeu.
- #define SHIP_SIZE 32

  Taille d'un vaisseau.
- #define METEORITE_SIZE 32

  Taille d'un météorite.
- #define FINISH_LINE_HEIGHT -960

  Hauteur de la ligne d'arrivée.
- #define MOVING_STEP 10

  Pas de déplacement horizontal du vaisseau.
- #define INITIAL_SPEED 1

  Vitesse initiale de déplacement vertical des éléments du jeu.

4.1.1 Detailed Description

Contient l'ensemble des constantes du jeu.

Author

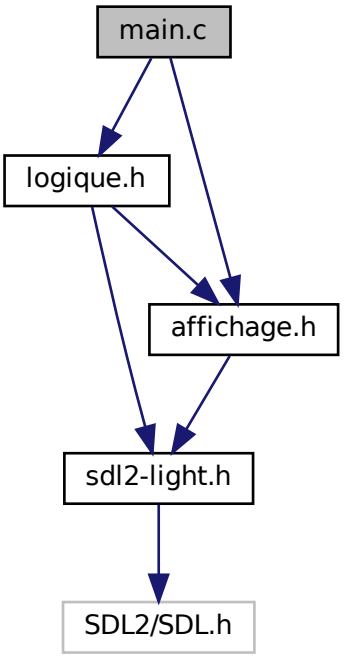
Valentini Andréa & Viale Romain

4.2 main.c File Reference

Programme principal du jeu.

```
#include "logique.h"
#include "affichage.h"
```

Include dependency graph for main.c:



Functions

- void clean (SDL_Window ∗window, SDL_Renderer ∗renderer, textures_t ∗textures, world_t ∗world)

  fonction qui nettoie le jeu: nettoyage de la partie graphique (SDL), nettoyage des textures, nettoyage des données
- void print_sprite (sprite_t ∗sprite)

  Fonction de test des coordonnées du sprite.
- int main (int argc, char ∗args[ ])

  programme principal qui implémente la boucle du jeu

4.2.1 Detailed Description

Programme principal du jeu.

Author

Valentini Andréa & Viale Romain

4.2.2 Function Documentation

4.2.2.1 clean()

```
void clean (
            SDL_Window * window,
            SDL_Renderer * renderer,
            textures_t * textures,
            world_t * world )
```

fonction qui nettoie le jeu: nettoyage de la partie graphique (SDL), nettoyage des textures, nettoyage des données

Parameters

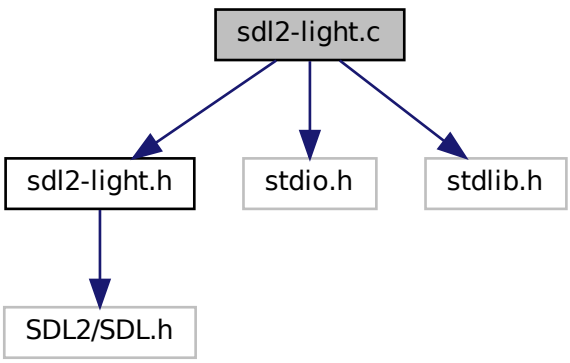
window	la fenêtre du jeu
renderer	le renderer
textures	les textures
world	le monde

4.3 sdl2-light.c File Reference

sur-couche de SDL2 pour simplifier son utilisation pour le projet

```
#include "sdl2-light.h"
#include <stdio.h>
#include <stdlib.h>
```

Include dependency graph for sdl2-light.c:



Functions

- int init_sdl (SDL_Window **window, SDL_Renderer **renderer, int width, int height)

  La fonction initialise la SDL. Elle crée la fenêtre du jeu ainsi que le renderer.
- SDL_Texture * load_image (const char path[ ], SDL_Renderer *renderer)

  La fonction charge une image et renvoie la texture correspondante où la couleur RGB (255, 0, 255) est rendue transparente.
- void apply_texture (SDL_Texture *texture, SDL_Renderer *renderer, int x, int y)

  La fonction permet d'appliquer une texture sur le renderer à une position donnée. La hauteur et la largeur est la même que celle de la texture.
- void clean_texture (SDL_Texture *texture)

  La fonction nettoie une texture en mémoire.
- void clear_renderer (SDL_Renderer *renderer)

  La fonction vide le contenu graphique du renderer lié à l'écran de jeu.
- void update_screen (SDL_Renderer *renderer)

  La fonction met à jour l'écran avec le contenu du renderer.
- void pause (int time)

  La fonction met le programme en pause pendant un laps de temps.
- void clean_sdl (SDL_Renderer *renderer, SDL_Window *window)

  La fonction nettoie le renderer et la fenêtre du jeu en mémoire.

4.3.1 Detailed Description

sur-couche de SDL2 pour simplifier son utilisation pour le projet

Author

Mathieu Constant

Version

0.2

Date

10 mars 2021

4.3.2 Function Documentation

4.3.2.1 apply_texture()

```
void apply_texture (
            SDL_Texture * texture,
            SDL_Renderer * renderer,
            int x,
            int y )
```

La fonction permet d'appliquer une texture sur le renderer à une position donnée. La hauteur et la largeur est la même que celle de la texture.

Parameters

texture	la texture que l'on va appliquer
renderer	le renderer qui va recevoir la texture
x	l'abscisse sur le renderer de l'endroit où est appliquée texture (point en haut à gauche de la surface)
y	l'ordonnée sur le renderer de l'endroit où est appliquée texture (point en haut à gauche de la surface)

4.3.2.2 clean_sdl()

```
void clean_sdl (
            SDL_Renderer * renderer,
            SDL_Window * window )
```

La fonction nettoie le renderer et la fenêtre du jeu en mémoire.

Parameters

renderer	le renderer à nettoyer
window	la fenêtre à nettoyer

4.3.2.3 clean_texture()

```
void clean_texture (
            SDL_Texture * texture )
```

La fonction nettoie une texture en mémoire.

Parameters

texture	la texture à nettoyer

4.3.2.4 clear_renderer()

```
void clear_renderer (
            SDL_Renderer * renderer )
```

La fonction vide le contenu graphique du renderer lié à l'écran de jeu.

Parameters

renderer	le renderer de l'écran

4.3.2.5 init_sdl()

```
int init_sdl (
             SDL_Window ** window,
             SDL_Renderer ** renderer,
             int width,
             int height )
```

La fonction initialise la SDL. Elle crée la fenêtre du jeu ainsi que le renderer.

Parameters

window	la fenêtre du jeu
renderer	le renderer
width	largeur de l'écran de jeu
height	hauteur de l'écran de jeu

Returns

-1 en cas d'erreur, 0 sinon

4.3.2.6 load_image()

```
SDL_Texture* load_image (
             const char path[],
             SDL_Renderer * renderer )
```

La fonction charge une image et renvoie la texture correspondante où la couleur RGB (255, 0, 255) est rendue transparente.

Parameters

path	est le chemin du fichier image. Le fichier doit être obligatoirement du BMP.
renderer	le renderer

Returns

la surface SDL contenant l'image avec la couleur RGB (255,0,255) rendue transparente. Elle renvoie NULL si le chargement a échoué (ex. le fichier path n'existe pas)

4.3.2.7 pause()

```
void pause (
             int time )
```

La fonction met le programme en pause pendant un laps de temps.

Parameters

time	ce laps de temps en milliseconde

4.3.2.8 update_screen()

```
void update_screen (
            SDL_Renderer * renderer )
```

La fonction met à jour l'écran avec le contenu du renderer.

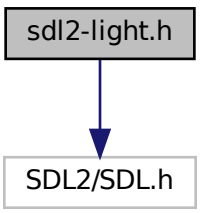
Parameters

renderer	le renderer de l'écran

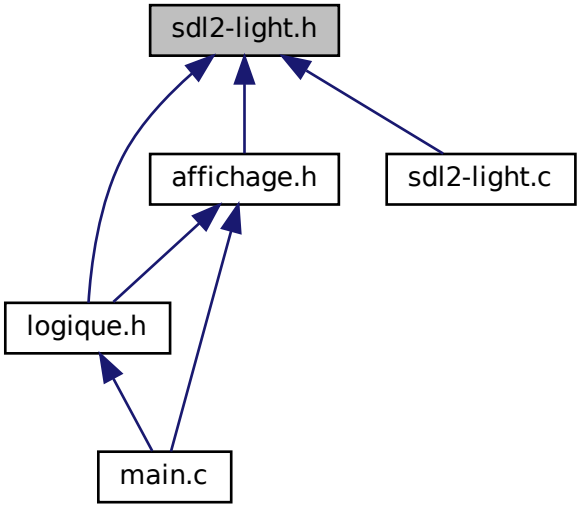
4.4 sdl2-light.h File Reference

en-tête du module correspondant à une sur-couche de SDL2 pour simplifier son utilisation pour le projet

```
#include <SDL2/SDL.h>
```
Include dependency graph for sdl2-light.h:



This graph shows which files directly or indirectly include this file:



Functions

- void clean_sdl (SDL_Renderer ∗renderer, SDL_Window ∗window)

  La fonction nettoie le renderer et la fenêtre du jeu en mémoire.
- SDL_Texture ∗ load_image (const char path[ ], SDL_Renderer ∗renderer)

  La fonction charge une image et renvoie la texture correspondante où la couleur RGB (255, 0, 255) est rendue transparente.
- int init_sdl (SDL_Window ∗∗window, SDL_Renderer ∗∗renderer, int width, int height)

  La fonction initialise la SDL. Elle crée la fenêtre du jeu ainsi que le renderer.
- void clean_texture (SDL_Texture ∗texture)

  La fonction nettoie une texture en mémoire.
- void apply_texture (SDL_Texture ∗texture, SDL_Renderer ∗renderer, int x, int y)

  La fonction permet d'appliquer une texture sur le renderer à une position donnée. La hauteur et la largeur est la même que celle de la texture.
- void clear_renderer (SDL_Renderer ∗renderer)

  La fonction vide le contenu graphique du renderer lié à l'écran de jeu.
- void update_screen (SDL_Renderer ∗renderer)

  La fonction met à jour l'écran avec le contenu du renderer.
- void pause (int time)

  La fonction met le programme en pause pendant un laps de temps.

4.4.1 Detailed Description

en-tête du module correspondant à une sur-couche de SDL2 pour simplifier son utilisation pour le projet

Author

Mathieu Constant

Version

0.2

Date

10 mars 2021

4.4.2 Function Documentation

4.4.2.1 apply_texture()

```
void apply_texture (
            SDL_Texture * texture,
            SDL_Renderer * renderer,
            int x,
            int y )
```

La fonction permet d'appliquer une texture sur le renderer à une position donnée. La hauteur et la largeur est la même que celle de la texture.

Parameters

texture	la texture que l'on va appliquer
renderer	le renderer qui va recevoir la texture
x	l'abscisse sur le renderer de l'endroit où est appliquée texture (point en haut à gauche de la surface)
y	l'ordonnée sur le renderer de l'endroit où est appliquée texture (point en haut à gauche de la surface)

4.4.2.2 clean_sdl()

```
void clean_sdl (
            SDL_Renderer * renderer,
            SDL_Window * window )
```

La fonction nettoie le renderer et la fenêtre du jeu en mémoire.

Parameters

renderer	le renderer à nettoyer
window	la fenêtre à nettoyer

4.4.2.3 clean_texture()

```
void clean_texture (
            SDL_Texture * texture )
```

La fonction nettoie une texture en mémoire.

Parameters

texture	la texture à nettoyer

4.4.2.4 clear_renderer()

```
void clear_renderer (
            SDL_Renderer * renderer )
```

La fonction vide le contenu graphique du renderer lié à l'écran de jeu.

Parameters

renderer	le renderer de l'écran

4.4.2.5 init_sdl()

```
int init_sdl (
            SDL_Window ** window,
            SDL_Renderer ** renderer,
            int width,
            int height )
```

La fonction initialise la SDL. Elle crée la fenêtre du jeu ainsi que le renderer.

Parameters

window	la fenêtre du jeu
renderer	le renderer
width	largeur de l'écran de jeu
height	hauteur de l'écran de jeu

Returns

-1 en cas d'erreur, 0 sinon

4.4.2.6 load_image()

```
SDL_Texture* load_image (
             const char path[],
             SDL_Renderer * renderer )
```

La fonction charge une image et renvoie la texture correspondante où la couleur RGB (255, 0, 255) est rendue transparente.

Parameters

path	est le chemin du fichier image. Le fichier doit être obligatoirement du BMP.
renderer	le renderer

Returns

la surface SDL contenant l'image avec la couleur RGB (255,0,255) rendue transparente. Elle renvoie NULL si le chargement a échoué (ex. le fichier path n'existe pas)

4.4.2.7 pause()

```
void pause (
             int time )
```

La fonction met le programme en pause pendant un laps de temps.

Parameters

time	ce laps de temps en milliseconde

4.4.2.8 update_screen()

```
void update_screen (
             SDL_Renderer * renderer )
```

La fonction met à jour l'écran avec le contenu du renderer.

Parameters

renderer	le renderer de l'écran

Index

apply_texture
    sdl2-light.c, 14
    sdl2-light.h, 19

background
    textures_s, 6

clean
    main.c, 13
clean_sdl
    sdl2-light.c, 15
    sdl2-light.h, 19
clean_texture
    sdl2-light.c, 15
    sdl2-light.h, 20
clear_renderer
    sdl2-light.c, 15
    sdl2-light.h, 20
constantes.h, 11

gameover
    world_s, 8

h
    sprite_s, 5

init_sdl
    sdl2-light.c, 16
    sdl2-light.h, 20

ligne_arrivee
    textures_s, 6
    world_s, 8
load_image
    sdl2-light.c, 16
    sdl2-light.h, 21

main.c, 12
    clean, 13
meteorite
    textures_s, 7

pause
    sdl2-light.c, 16
    sdl2-light.h, 21

sdl2-light.c, 13
    apply_texture, 14
    clean_sdl, 15
    clean_texture, 15
    clear_renderer, 15
    init_sdl, 16
    load_image, 16
    pause, 16
    update_screen, 17
sdl2-light.h, 17
    apply_texture, 19
    clean_sdl, 19
    clean_texture, 20
    clear_renderer, 20
    init_sdl, 20
    load_image, 21
    pause, 21
    update_screen, 21
sprite_s, 5
    h, 5
    w, 5
    x, 5
    y, 6

textures_s, 6
    background, 6
    ligne_arrivee, 6
    meteorite, 7
    vaisseau, 7
time
    world_s, 8

update_screen
    sdl2-light.c, 17
    sdl2-light.h, 21

vaisseau
    textures_s, 7
    world_s, 8
vy
    world_s, 8

w
    sprite_s, 5
wall
    world_s, 8
win
    world_s, 9
world_s, 7
    gameover, 8
    ligne_arrivee, 8
    time, 8
    vaisseau, 8
    vy, 8
    wall, 8

win, 9

x
sprite_s, 5

y
sprite_s, 6